LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 5

CONNECT TO THE INTERNET

Oleh:

Bi'ahlil Haq Aulia Akbar Awaludin NIM. 2010817110011

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2025

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN II
MODUL 5

Laporan Praktikum Pemrograman Mobile Modul 5: Connect to the Internet ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Bi'ahlil Haq Aulia Akbar Awaludin
NIM : 2010817110011

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Raka Azwar NIM. 2210817210012	Ir. EKA SETYA WIJAYA, S.T., M.Kom NIP. 19820508 200801 10 10

DAFTAR ISI

DAFTAR GAMBAR

DAFTAR TABEL

SOAL 1

Soal Praktikum:

1. Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:
    a. Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
    b. Gunakan KotlinX Serialization sebagai library JSON.
    c. Gunakan library seperti Coil atau Glide untuk image loading.
    d. API yang digunakan pada modul ini adalah The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: https://developer.themoviedb.org/docs/getting-started
    e. Implementasikan konsep data persistence (aplikasi menyimpan data walau pengguna keluar dari aplikasi) dengan SharedPreferences untuk menyimpan data ringan (seperti pengaturan aplikasi) dan Room untuk data relasional.
    f. Gunakan caching strategy pada Room. Dibebaskan untuk memilih caching strategy yang sesuai, dan sertakan penjelasan kenapa menggunakan caching strategy tersebut.
    g. Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.
2. Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code

Compose
AppModule.kt

```
1   package com.example.movielist.di
2
3   import android.content.Context
4   import androidx.room.Room
5   import com.example.movielist.data.ktor.ApiService
6   import com.example.movielist.data.ktor.ApiServiceImpl
7   import com.example.movielist.data.ktor.KtorClientProvider
8   import com.example.movielist.data.local.MovieDao
9   import com.example.movielist.data.local.MovieDatabase
10  import com.example.movielist.data.repository.MovieRepository
11  import
12  com.example.movielist.data.repository.fake.FakeMovieReposito
13  ry
14  import
15  com.example.movielist.data.repository.MovieRepositoryImpl
16  import dagger.Module
17  import dagger.Provides
18  import dagger.hilt.InstallIn
19  import dagger.hilt.android.qualifiers.ApplicationContext
20  import dagger.hilt.components.SingletonComponent
21  import io.ktor.client.HttpClient
22  import javax.inject.Singleton
23
24
25  @Module
26  @InstallIn(SingletonComponent::class)
27  object AppModule {
28
29      @Provides
30      @Singleton
31      fun provideKtorClient(): HttpClient {
32          return KtorClientProvider.httpClient
33      }
34
35      @Provides
36      @Singleton
37      fun provideApiService(client: HttpClient): ApiService {
38          return ApiServiceImpl(client)
39      }
40
41      @Provides
42      @Singleton
43        fun provideMovieDatabase(@ApplicationContext context:
44  Context): MovieDatabase {
45          return Room.databaseBuilder(
46              context,
47              MovieDatabase::class.java,
```

```
48              "movie_database"
49          ).fallbackToDestructiveMigration().build()
50      }
51
52      @Provides
53      @Singleton
54      fun provideMovieDao(database: MovieDatabase): MovieDao {
55          return database.movieDao()
56      }
57
58      @Provides
59      @Singleton
60       fun provideMovieRepository(@ApplicationContext context:
61  Context, api: ApiService, dao: MovieDao): MovieRepository {
62          // Untuk menggunakan repository asli (dengan Ktor)
63          return MovieRepositoryImpl(api, dao)
64
65              // Beri komentar pada baris di atas dan hapus
66  komentar di bawah ini
67  //          return FakeMovieRepository(context = context)
68      }
69  }
```

Tabel 1. Source Code Jawaban Soal 1

KtorClientProvider.kt

```
1   package com.example.movielist.data.ktor
2
3   import io.ktor.client.*
4   import io.ktor.client.engine.cio.*
5   import io.ktor.client.plugins.contentnegotiation.*
6   import io.ktor.client.plugins.defaultRequest
7   import io.ktor.serialization.kotlinx.json.*
8   import io.ktor.util.appendIfNameAbsent
9   import kotlinx.serialization.json.Json
10
11  object KtorClientProvider {
12      val httpClient: HttpClient by lazy {
13          HttpClient(CIO) {
14              install(ContentNegotiation) {
15                  json(
16                      Json {
17                          ignoreUnknownKeys = true
18                      }
19                  )
20              }
21              engine {
22                  requestTimeout = 15_000
23                  endpoint {
24                      connectTimeout = 15_000
25                      socketTimeout = 15_000
26                  }
```

```
            }
            // Configure default request parameters
            defaultRequest {
                 url("https://api.themoviedb.org")
                        headers.appendIfNameAbsent("Accept",
"application/json")
                 headers.appendIfNameAbsent("Authorization",
"Bearer
eyJhbGciOiJIUzI1NiJ9.eyJhdWQiOiIwZGJkZGQ5MjBlM2FkNDU0M2RkN2Z
lYjNmOWI2YzViMCIsIm5iZiI6MTc0OTAxODExNS44NzYsInN1YiI6IjY4M2Z
lNjAzMDUzNjE5YTdhZGZkYmVlMCIsInNjb3BlcyI6WyJhcGlfcmVhZCJdLCJ
2ZXJzaW9uIjoxfQ.BBXzrwJuk_3W0YFiosG7wZ9uP_Ll5Zi93ZpQBd4M7iA"
)
            }
            // Configure client-wide settings
            expectSuccess = true
            followRedirects = true
        }
    }
}
```

Tabel 2. Source Code Jawaban Soal 1

ApiServiceImpl.kt

```
package com.example.movielist.data.ktor

import com.example.movielist.data.ktor.dto.MovieDetailDto
import
com.example.movielist.data.ktor.dto.MovieListResponseDto

import io.ktor.client.*
import io.ktor.client.call.*
import io.ktor.client.plugins.*
import io.ktor.client.request.*
import io.ktor.http.*
import timber.log.Timber

class ApiServiceImpl(
    private val client: HttpClient
) : ApiService {

            override suspend fun getDiscoverMovies():
MovieListResponseDto {
        return try {

            client.get("/3/discover/movie") {
                url {
                        parameters.append("include_adult",
"false")

```

```
                            parameters.append("include_video",
"false")
                        parameters.append("language", "en-US")
                                    parameters.append("sort_by",
"popularity.desc")
                            parameters.append("include_video",
"false")
                        parameters.append("with_genres", "16")
                    }
                    accept(ContentType.Application.Json)
                            Timber.tag("ApiService").d("Berhasil
mengambil daftar film")
                }.body()

            } catch (e: Exception) {
                      Timber.tag("ApiService").e(e, "Error saat
mengambil daftar film")
                MovieListResponseDto(emptyList())
            }
        }


              override suspend fun getMovieDetail(id: Int):
MovieDetailDto? {
            return try {
                client.get("/3/movie/$id") {
                    url {
                        parameters.append("language", "en-US")
                    }
                    accept(ContentType.Application.Json)
                }.body()
            } catch (e: ClientRequestException) {
                // 4xx errors
                                    Timber.tag("ApiService").e(e,
"ClientRequestException")
                null
            } catch (e: Exception) {
                      Timber.tag("ApiService").e(e, "Error saat
mengambil detail film")
                null
            }
        }
}
```

Tabel 3. Source Code Jawaban Soal 1

MovieDao.kt

```
package com.example.movielist.data.local

import androidx.room.*
```

```
4   import kotlinx.coroutines.flow.Flow
5
6   @Dao
7   interface MovieDao {
8
9       @Query("SELECT * FROM movies WHERE id = :id")
10      fun getMovieById(id: Int): Flow<MovieEntity?>
11
12      @Query("SELECT * FROM movies")
13      fun getAllMovies(): Flow<List<MovieEntity>>
14
15      @Insert(onConflict = OnConflictStrategy.REPLACE)
16      suspend fun insertAll(movies: List<MovieEntity>)
17
18      @Query("DELETE FROM movies")
19      suspend fun clearAll()
20
21        @Query("UPDATE movies SET imdbId = :imdbId WHERE id =
22  :movieId")
23      suspend fun updateImdbId(movieId: Int, imdbId: String)
24  }
```

Tabel 4. Source Code Jawaban Soal 1

MovieEntity.kt

```
1   package com.example.movielist.data.local
2
3   import androidx.room.Entity
4   import androidx.room.PrimaryKey
5
6   @Entity(tableName = "movies")
7   data class MovieEntity(
8       @PrimaryKey
9       val id: Int,
10      val title: String,
11      val overview: String,
12      val posterPath: String,
13      val releaseDate: String,
14      val imdbId: String? = null
15  )
```

Tabel 5. Source Code Jawaban Soal 1

Mappers.kt

```
1   package com.example.movielist.data
2
3   import com.example.movielist.data.ktor.dto.MovieFromListDto
4   import com.example.movielist.data.local.MovieEntity
5   import com.example.movielist.domain.model.Movie
```

```
6
7   // Mengubah dari Network Model ke Entity (database)
8   fun MovieFromListDto.toEntity(): MovieEntity {
9       return MovieEntity(
10          id = this.id,
11          title = this.title ?: "Unknown Title",
12                  overview = this.overview ?: "No overview
13  available.",
14          posterPath = this.posterPath ?: "",
15          releaseDate = this.releaseDate ?: "Unknown Date"
16      )
17  }
18
19  // Mengubah dari Entity (database) ke Domain Model (untuk
20  UI)
21  fun MovieEntity.toDomainMovie(): Movie {
22      val imdbUrl = if (this.imdbId?.isNotEmpty() == true) {
23          "https://www.imdb.com/title/${this.imdbId}/"
24      } else {
25          ""
26      }
27
28      return Movie(
29          id = this.id.toString(),
30          title = this.title,
31          description = this.overview,
32                imageUrl = if (this.posterPath.isNotEmpty())
33  "https://image.tmdb.org/t/p/original${this.posterPath}" else
34  "",
35           year = this.releaseDate.split("-").firstOrNull() ?:
36  "N/A",
37          imdbURL = imdbUrl
        )
    }
```

Tabel 6. Source Code Jawaban Soal 1

MovieRepositoryImpl.kt

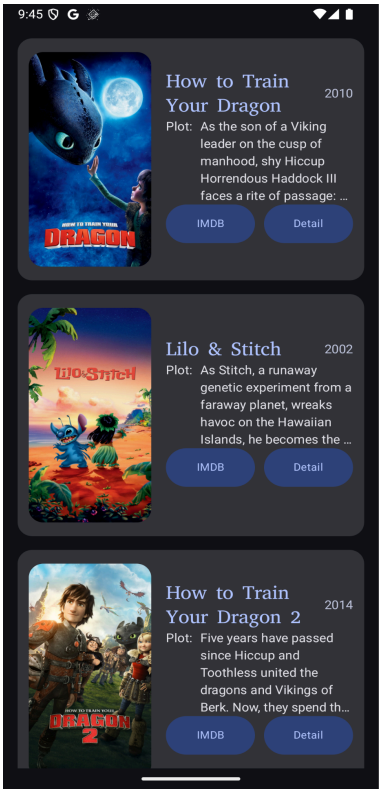
```
1   package com.example.movielist.data.repository
2
3   import com.example.movielist.data.Result
4   import com.example.movielist.data.ktor.ApiService
5   import com.example.movielist.data.local.MovieDao
6   import com.example.movielist.data.toDomainMovie
7   import com.example.movielist.data.toEntity
8   import com.example.movielist.domain.model.Movie
9   import kotlinx.coroutines.coroutineScope
10  import kotlinx.coroutines.flow.Flow
11  import kotlinx.coroutines.flow.channelFlow
12  import kotlinx.coroutines.flow.flow
```

```
import kotlinx.coroutines.launch
import timber.log.Timber
import javax.inject.Singleton

@Singleton
class MovieRepositoryImpl (
    private val api: ApiService,
    private val dao: MovieDao
) : MovieRepository {

    override fun getAllMovies(): Flow<Result<List<Movie>>> =
channelFlow {
        send(Result.Loading(true))

        try {
                                        val   remoteMovies   =
api.getDiscoverMovies().results
                    val  movieEntities  =  remoteMovies.map  {
it.toEntity() }

Timber.Forest.tag("MovieRepositoryImpl").d("getDiscoverMovie
s: $remoteMovies")
            dao.clearAll()
            dao.insertAll(movieEntities)

Timber.Forest.tag("MovieRepositoryImpl").d("insertAll:
$movieEntities")

            coroutineScope {
                movieEntities.forEach { movie ->
                    launch {
                                                val  detail  =
api.getMovieDetail(movie.id)

Timber.Forest.tag("MovieRepositoryImpl").d("getMovieDetail:
$detail")
                        if (detail?.imdbId != null) {
                                dao.updateImdbId(movie.id,
detail.imdbId)
                        }
                    }
                }
            }
        } catch (e: Exception) {
            send(Result.Error(e))
        } finally {
            send(Result.Loading(false))
        }
        dao.getAllMovies().collect { movies ->

Timber.Forest.tag("MovieRepositoryImpl").d("getAllMovies:
$movies")
```

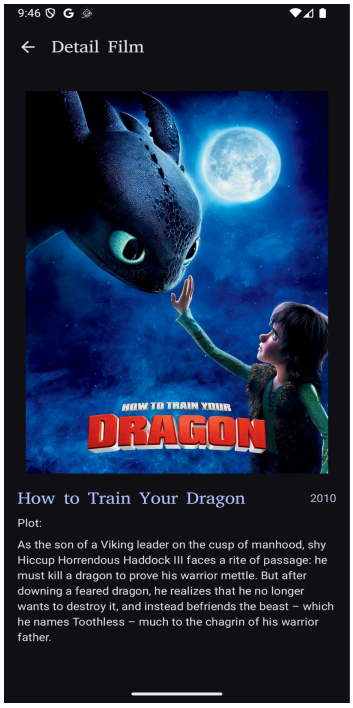
```
65                                  send(Result.Success(movies.map  {
66  it.toDomainMovie() }))
67              }
68          }
69
70          override fun getMovie(postId: Int?): Flow<Result<Movie>>
71  {
72              return flow {
73                  emit(Result.Loading(true))
74                  if (postId == null) {
75                       emit(Result.Error(Exception("Movie ID tidak
76  valid.")))
77                      return@flow
78                  }
79                  try {
80                      dao.getMovieById(postId).collect { entity ->
81                          if (entity != null) {
82
83  emit(Result.Success(entity.toDomainMovie()))
84                          } else {
85                                  emit(Result.Error(Exception("Film
86  dengan ID $postId tidak ditemukan di cache.")))
87                          }
88                      }
89                  } catch (e: Exception) {
90                      emit(Result.Error(e))
91                  } finally {
92                      emit(Result.Loading(false))
93
94                  }
95              }
96          }
97  }
```

Tabel 7. Source Code Jawaban Soal 1

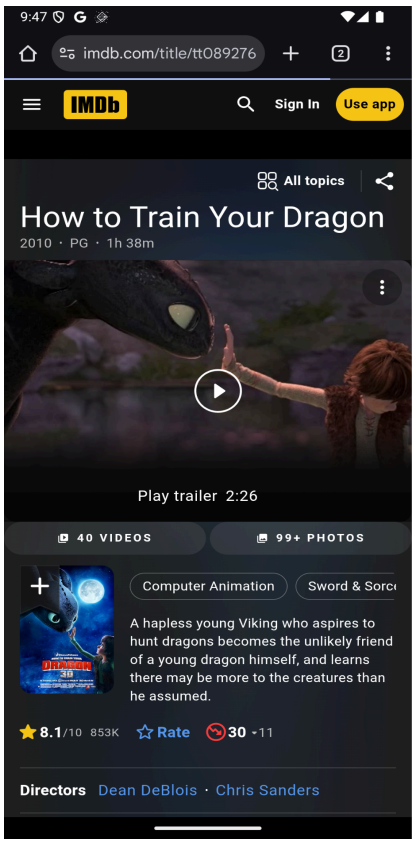
B. Output Program



Gambar 1. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 2. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 3. Screenshot Hasil Jawaban Soal 1 Compose

C. Pembahasan

Compose

AppModule.kt:

Adalah tempat dimana semua dependensi injection provider akan diletakan. Kenapa pakai provider? Karena hampir semuanya menggunakan interface, jadi binding redundant.

KtorClientProvider.kt:

Membuat class client yang berisi config dan default request template yang akan link ke dependensi injection dan bisa digunakan oleh banyak versi dari implementasi api.

ApiServiceImpl.kt:

Tempat membuat request kepada api seperti pada `getDiscoverMovies` dimana melanjutkan config dari client provider.

MovieDao.kt:

Data Access Object adalah interface dimana anda dapat melakukan query kedalam database yang akan dibuat oleh room.

MovieEntity.kt:

Movie Entity merupakan data class model atau table pada database.

Mappers.kt:

Mappers adalah tempat menempatkan global fungsi yang bekerja untuk merubah dan mengatur data dari network ke local ke model ui.

MovieRepositoryImpl.kt:

Tempat dimana implementasi dari apiservice, dan room. Data dari sini akan di teruskan kepada view model untuk dijalankan. Repository akan melakukan emit state.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.
https://github.com/biahlil/Pemrograman-Mobile.git